

MODULE 06 · DOMAIN 3 - APPLICATIONS OF FOUNDATION MODELS

RAG & Knowledge Bases

Grounding a model in your own data — and knowing when not to.

DOMAIN 3 of 5 EXAM WEIGHT 28% MODULE 6 of 14

RAG does not change the model. It changes **what the model can see** at the moment you ask.

Five stages, and the exam asks about stage two

01

Chunk

Split documents into passages small enough to retrieve usefully.

02

Embed

Convert each chunk into a vector using an embedding model.

03

Store

Index those vectors in a vector database.

04

Retrieve

Embed the query, find the nearest chunks by similarity.

05

Augment

Inject retrieved chunks into the prompt, then generate.

ON THE EXAM

When a question says a company wants to use its own documents as a knowledge base, the correct option is almost always the one mentioning embeddings created from document chunks.

What RAG buys you

- ◆ **Grounding.** Answers tie to retrieved passages. The standard fix for hallucination.
- ◆ **Freshness.** Update the documents and answers update. No retraining cycle.
- ◆ **Citations.** You can show which passage produced the answer, supporting transparency.
- ◆ **No labelled data needed.** Works when you have documents but not training examples.
- ◆ **Access control survives.** Retrieval can respect who is allowed to see what.

ON THE EXAM

If a scenario says labelled data is limited or unavailable, that is a strong signal for RAG over fine-tuning.

RAG or fine-tuning?

CHOOSE RAG WHEN

The problem is knowledge

- The model lacks facts, not skills
- Content changes often
- You have documents, not labelled examples
- You need citations or auditability
- Answers must reflect current company data

CHOOSE FINE-TUNING WHEN

The problem is behaviour

- You need a specific tone, format or style
- A specialised task the base model does poorly
- You have a high-quality labelled dataset
- Latency matters and retrieval is too slow
- The behaviour must hold without extra context

THE TIEBREAKER

Knowledge problem, use RAG. Behaviour problem, fine-tune. If the scenario mentions a labelled dataset, that phrase is pointing at fine-tuning.

Chunking is where quality is won or lost

- ◆ Too small and a chunk loses the context that made it meaningful.
- ◆ Too large and the match dilutes, and you burn tokens on irrelevant text.
- ◆ **Overlap** between adjacent chunks stops an answer being split at a boundary.
- ◆ Bedrock offers fixed-size, hierarchical and semantic chunking, or none at all.
- ◆ Semantic chunking splits on meaning rather than character count.

ON THE EXAM

Chunk size has no universally correct value. It depends on document structure, and the exam accepts it as a tuning decision.

Embeddings, and the rule people break

- ◆ An embedding model turns text into a vector. Titan Embeddings and Cohere Embed are the usual choices.
- ◆ **Query and documents must use the same embedding model.** Different models produce incomparable vectors.
- ◆ Change the embedding model and you must **re-index everything**.
- ◆ Multimodal embedding models let you retrieve across images and documents together.

ON THE EXAM

Retrieval returning irrelevant results after a model change is almost always a mismatched embedding model.

How retrieval actually picks passages

- ◆ **Semantic search** matches meaning. It finds relevant text sharing no words with the query.
- ◆ **Keyword search** matches exact terms. Better for product codes, names and identifiers.
- ◆ **Hybrid search** combines both, and is usually the strongest default.
- ◆ **Top-k** sets how many chunks are retrieved and passed to the model.
- ◆ **Reranking** reorders retrieved chunks so the best land nearest the question.

ON THE EXAM

Semantic search is why RAG beats a keyword search box. Hybrid is the answer when exact identifiers also matter.

The managed version of all of that

- ◆ Point it at a data source and it handles chunking, embedding, storage and retrieval.
- ◆ Sources include S3, SharePoint, Salesforce, Confluence and web crawlers.
- ◆ **RetrieveAndGenerate** returns an answer with citations. **Retrieve** returns passages only.
- ◆ **Metadata filtering** restricts retrieval by attributes such as department or date.
- ◆ Syncing re-indexes changed documents on a schedule or on demand.

ON THE EXAM

Knowledge Bases is the answer whenever a scenario wants RAG without building the pipeline. It is managed RAG, nothing more mysterious.

Where the vectors actually live

STORE	CHOOSE IT WHEN
OpenSearch Serverless	The default. Quick-create, nothing to manage.
Aurora PostgreSQL	You already run Postgres and want pgvector beside relational data.
Neptune Analytics	Relationships between entities matter. The GraphRAG option.
S3 Vectors	Large vector sets where cost per vector outweighs latency.
Pinecone / MongoDB / Redis	Third-party stores you already operate.

ON THE EXAM

You will not be asked to configure these. You may be asked which suits graph-shaped data — that is Neptune Analytics.

When RAG gives wrong answers

- ◆ **Suspect retrieval first.** A bad answer usually means the right passage was never fetched.
- ◆ Check chunk size, then overlap, then whether the embedding model matches.
- ◆ Check top-k. Too low and the relevant chunk falls outside the window.
- ◆ Instruct the model to answer **only** from retrieved context, and to say when it cannot.
- ◆ Swapping to a larger generation model rarely fixes a retrieval problem.

ON THE EXAM

Bad retrieval looks exactly like a bad model. The distinguishing test is whether the correct passage appears in what was retrieved.

RAG or an agent?

RAG

Look it up, then answer

- Read-only retrieval from a document store
- One retrieval, one generation
- Answers questions
- Nothing changes in any system

AGENTS

Decide, then act

- Calls APIs and external services
- Multiple steps, chosen at runtime
- Completes transactions
- State changes as a result

THE TIEBREAKER

If the scenario says the system must invoke an API, complete a booking, or update a record, it is an agent — not RAG.

Two risks specific to RAG

- ◆ **Over-retrieval.** If the index ignores permissions, users see documents they should not.
- ◆ Metadata filtering and permission-aware retrieval are the controls.
- ◆ **Indirect prompt injection.** A malicious instruction hidden inside a retrieved document.
- ◆ Retrieved content is untrusted input. Treat it as data, never as instructions.

ON THE EXAM

Indirect injection is the risk people miss. In RAG, the attack does not have to come from the user typing.

RAG or just a bigger context window?

LONG CONTEXT

Put everything in the prompt

- Simple. No pipeline to build
- Works when the corpus is genuinely small
- You pay for every token, on every call
- Quality degrades as the context fills

RAG

Fetch only what is needed

- Scales to millions of documents
- You pay only for retrieved passages
- Citations come for free
- Adds retrieval infrastructure

THE TIEBREAKER

A million-token window does not make RAG obsolete. It makes a small corpus cheap to handle and a large one ruinous.

Measure retrieval and generation separately

- ◆ **Retrieval quality** — was the correct passage among the chunks fetched?
- ◆ **Generation quality** — given those chunks, was the answer faithful to them?
- ◆ Measuring only the final answer hides **which half** is broken.
- ◆ **Faithfulness** asks whether the answer is supported by the retrieved text.
- ◆ **Relevance** asks whether it actually answered the question asked.

ON THE EXAM

A system can retrieve perfectly and still answer badly, or retrieve badly and sound convincing. One combined score tells you neither.

Where a RAG system actually spends money

- ◆ **Embedding** every document once at ingestion, and again on every change.
- ◆ **The vector store runs continuously**, so it costs money at zero traffic.
- ◆ **Retrieved chunks become prompt tokens**, so every answer costs more than the question alone.
- ◆ Still usually cheaper than fine-tuning, which adds provisioned throughput hosting.

ON THE EXAM

RAG's standing cost is the vector store. Fine-tuning's standing cost is model hosting. Neither is free when idle.

Carry these into the exam

- ◆ RAG changes context, not weights. Nothing is retrained.
- ◆ 'Embeddings created from document chunks' is the phrase to look for.
- ◆ Knowledge problem RAG. Behaviour problem fine-tune. 'Labelled dataset' means fine-tune.
- ◆ RAG is the standard answer for hallucination and grounding in company data.
- ◆ Query and documents must share an embedding model.
- ◆ Must call an API or change a record? That is an agent.

An airline's support assistant answers questions about baggage rules, which change several times a year and differ by route.

It currently gives outdated answers. The airline has current policy documents but no labelled question-and-answer data.

What should the airline implement?

- ☐ A Fine-tune the model on the policy documents
- ☐ B Implement RAG over the policy documents
- ☐ C Continued pre-training on aviation text
- ☐ D Increase the context window

◆ PAUSE AND DECIDE

What should the airline implement?

- A

 Fine-tune on the documents
Needs labelled pairs, and freezes rules that change several times a year.
- B

 Implement RAG
Grounds answers in current documents. Update the docs, update the answers.
- C

 Continued pre-training
Teaches vocabulary. The model understands the words already.
- D

 Larger context window
More room for text the model still does not have.

ON THE EXAM

Three signals agree: facts rather than behaviour, content that changes often, and no labelled data.

A manufacturer built a RAG assistant over its maintenance manuals. Answers are fluent but frequently wrong.

On inspection, the passages retrieved for each question are unrelated to the question asked. The team recently switched embedding models to reduce cost.

What is the most likely cause?

- ☐ A The generation model is too small
- ☐ B Temperature is set too high
- ☐ C The index was not rebuilt after the embedding model changed
- ☐ D The context window is too small

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

 Generation model too small
It cannot answer from passages that were never retrieved.
- B

 Temperature too high
Affects wording. Retrieval happens before generation.
- C

 Index not rebuilt
Query and stored vectors now come from different models.
- D

 Context window too small
Would truncate, not return unrelated passages.

ON THE EXAM

The question tells you retrieval returns unrelated passages. That locates the fault before generation, eliminating A, B and D at once.

A consultancy builds a Knowledge Base over its project files. Junior staff begin receiving answers drawn from partner-level compensation documents.

The underlying files have correct permissions in SharePoint.

What went wrong, and what fixes it?

- ☐ A Retrieval ignores permissions; apply metadata filtering
- ☐ B The model hallucinated; lower the temperature
- ☐ C Prompt injection; add Guardrails
- ☐ D Chunk size is too large; reduce it

◆ PAUSE AND DECIDE

What went wrong, and what fixes it?

- | | |
|---|--|
| <p>A Retrieval ignores permissions
<i>Indexing flattens access control unless you rebuild it in retrieval.</i></p> | <p>B Hallucination
<i>The content is real. It came from a genuine document.</i></p> |
| <p>C Prompt injection
<i>Nobody attacked anything. Staff asked ordinary questions.</i></p> | <p>D Chunk size
<i>Affects answer quality, not who can see which document.</i></p> |

ON THE EXAM

Source permissions do not survive indexing on their own. A vector does not carry a SharePoint access list with it.

A logistics firm wants an assistant that answers questions about shipping policy, and can also look up a live tracking number and reschedule a delivery.

Policy documents sit in S3. Tracking and rescheduling are available through internal APIs.

What architecture does this need?

- ☐ A RAG alone over the policy documents
- ☐ B Fine-tune a model on shipping data
- ☐ C An agent with action groups, plus a knowledge base
- ☐ D A larger context window holding all policies

◆ PAUSE AND DECIDE

What architecture does this need?

- A

 RAG alone
Covers the policy questions. Cannot call an API or reschedule.
- B

 Fine-tune on shipping data
Neither retrieves current policy nor performs actions.
- C

 Agent plus knowledge base
Retrieval for policy, action groups for tracking and rescheduling.
- D

 Larger context window
Still read-only, and rescheduling is not a reading problem.

ON THE EXAM

Watch for scenarios with two requirements. Rescheduling a delivery changes state, and only an agent can do that.